

PROYECTO FINAL

Sistema de gestión aeroportuaria

Programación orientada a objetos en Java · Técnica laboral

7

clases

5

consultas

2

integrantes



El encargo

Lo que pide la compañía de gestión aeroportuaria

Una compañía administra varios aeropuertos y necesita un programa para consultar la información de sus aeropuertos, las compañías aéreas que operan en cada uno, los vuelos que ofrecen y los pasajeros que han reservado asiento.



Aplicación de consola

Sin interfaz gráfica. Un menú de texto con cinco opciones.



Datos en memoria

Nada de base de datos ni archivos. Los datos se cargan al iniciar.



Java con POO

Clases, objetos, encapsulamiento, herencia y polimorfismo.

Lo que se evalúa de verdad

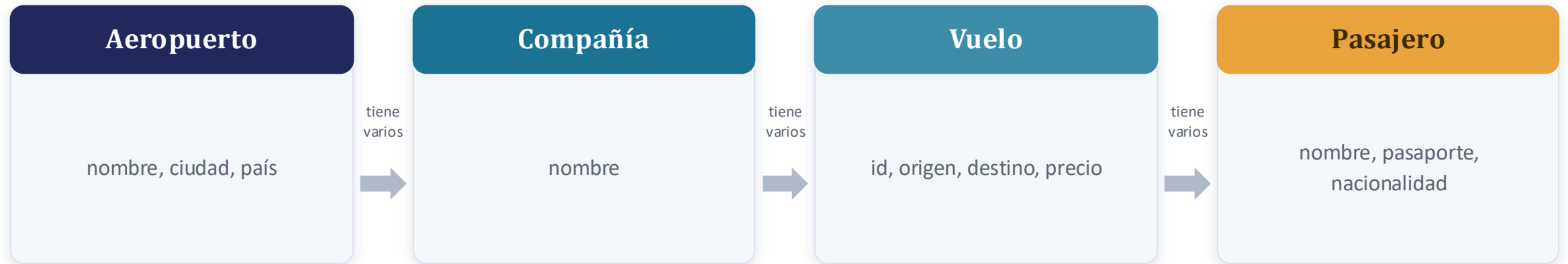
No se trata de escribir mucho código.

Se trata de que cada clase guarde lo que le corresponde y de que la herencia se use donde tiene sentido.

Un programa que funcione pero con todo metido en el main obtiene una nota baja.

La estructura, en una frase

Cada nivel contiene una lista del siguiente



Un aeropuerto **tiene varias compañías**, cada compañía **tiene varios vuelos**, y cada vuelo **tiene varios pasajeros**.

Esa cadena es la que va a recorrer con ciclos anidados en las opciones 4 y 5 del menú.

La herencia

Los aeropuertos son públicos o privados, nunca los dos



Reglas del negocio

Las condiciones que el programa debe respetar

1

Público o privado, nunca los dos

El público tiene subvención del gobierno; el privado tiene empresas patrocinadoras.

2

Un vuelo no acepta más pasajeros que su máximo

Si el vuelo está lleno, el programa avisa y no reserva.

3

El número real de pasajeros NO es un atributo

Se calcula con `pasajeros.size()`. Si lo guarda aparte, se le va a desincronizar.

4

Una compañía puede operar en varios aeropuertos

Ojo con esto en la opción 5: el mismo vuelo puede aparecer dos veces.

5

Todos los atributos son private

Se accede a ellos desde afuera únicamente con métodos `get`.

El menú de la aplicación

Cinco consultas, más la opción de salir

1

Aeropuertos gestionados

Públicos y privados por separado, con nombre, ciudad y país.

2

Financiación de un aeropuerto

Patrocinadores si es privado; subvención si es público.

3

Compañías de un aeropuerto

Lista de las compañías que vuelan desde ahí.

4

Vuelos de una compañía

Identificador, origen, destino y precio de cada vuelo.

5

Vuelos entre dos ciudades

El usuario indica origen y destino; se muestra id y precio.

0

Salir

Termina el programa.

La opción 2 es la clave de todo el proyecto

Las dos versiones funcionan. Solo una demuestra que entendió la herencia.

✗ Así NO

```
if (a instanceof AeropuertoPublico) {
    AeropuertoPublico p =
        (AeropuertoPublico) a;
    System.out.println(p.getSubvencion());
} else {
    AeropuertoPrivado r =
        (AeropuertoPrivado) a;
    // recorrer patrocinadores...
}
```

Si mañana aparece un tercer tipo de aeropuerto, hay que volver a tocar el menú.

✓ Así SÍ

```
// en Aeropuerto (clase padre)
public abstract String
    informacionDeFinanciacion();

// en el menú, una sola línea
System.out.println(
    a.informacionDeFinanciacion());
```

Cada aeropuerto sabe qué contestar. Eso es polimorfismo, y vale 15 puntos.

El menú no debería preguntar de qué tipo es un aeropuerto. Debería pedirle al aeropuerto que responda.

Requisitos técnicos obligatorios

Si falta alguno, el proyecto pierde puntos aunque funcione



ArrayList

Para todas las listas. Nada de arreglos de tamaño fijo.



Una clase por archivo

Con el mismo nombre de la clase y extensión .java.



Encapsulamiento

Atributos private, constructor y métodos get.



Herencia con super

Las dos hijas llaman a super(...) en la primera línea.



Método abstracto

Declarado en el padre, con @Override en las dos hijas.



Datos precargados

3 aeropuertos, 3 compañías, 6 vuelos y 5 pasajeros.



Menú a prueba de errores

Se repite hasta salir y no se cae con una opción inválida.



Encabezado con nombres

Cada opción imprime los nombres de los dos integrantes.

Cómo hacerlo, por fases

No empiece por el menú. Empiece por las clases y pruebe cada una.



Si llega el día de la entrega y solo tiene hasta la fase 4, entregue eso: vale mucho más que un menú a medio hacer sin clases bien construidas.

Cuatro errores que se cometen siempre

Revíselos antes de entregar



Guardar el número de pasajeros como atributo

Va a llegar el momento en que la lista tenga 3 y el contador diga 2.

Use `pasajeros.size()`



Resolver la opción 2 con instanceof

Funciona, pero es exactamente lo que la herencia viene a evitar.

Use un método abstracto



Vuelos repetidos en la opción 5

Si una compañía opera en dos aeropuertos, el mismo vuelo sale dos veces.

Decida qué hacer con el duplicado



Comparar textos con ==

Casi siempre da falso aunque el texto se vea igual en pantalla.

Use `equalsIgnoreCase()`

Qué se entrega

En parejas. Cada estudiante sube el archivo por separado, con los dos nombres.

1

Código fuente

.zip

Todos los archivos .java.
Si no compila, no se evalúa el funcionamiento.

2

Evidencias

PDF o Word

El código de cada clase y una captura de cada una de las cinco opciones.

3

Diagrama de clases

Imagen

Las 7 clases con atributos, métodos y las flechas de herencia. A mano también sirve.

4

Sustentación

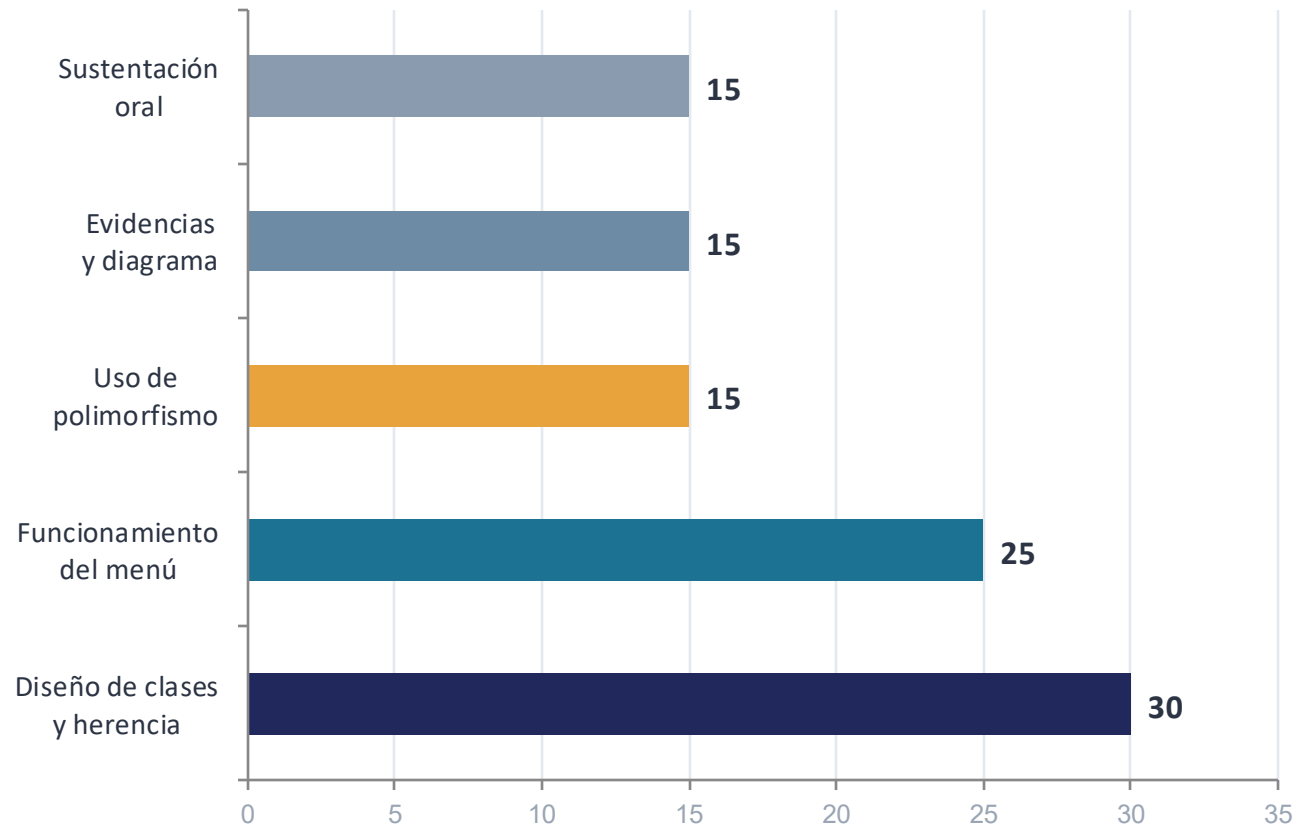
Oral, en clase

5 minutos por estudiante, explicando su propio código.

Un programa que funcione pero que su autor no pueda explicar no recibe la nota completa.

Cómo se califica

Sobre 100 puntos, que después se convierten a la escala de 1,0 a 5,0



45 de 100 puntos

dependen del diseño de clases y del uso del polimorfismo.

Es más de lo que vale que el menú funcione.

Un proyecto con las cinco opciones andando pero sin herencia real no llega a 3,0.

Un proyecto con las clases bien construidas y el menú incompleto sí aprueba.

Por dónde empezar hoy

- 1 **Formen la pareja** y descarguen la lista de chequeo.
- 2 **Escriban Pasajero.java** y prueben que guarda y devuelve los datos.
- 3 **Escriban Vuelo.java** y prueben el rechazo cuando está lleno.
- 4 **No abran el menú todavía** hasta que las clases funcionen.

Las clases primero. El menú es lo último y lo más fácil.

Fechas

Entrega del código
y las evidencias

Sustentación oral
